Servicio seguro de contenidos multimedia

Documentación del código — Fase I

Cifrado y descifrado de archivos con AES-128 y RSA-2048

Integrantes del grupo

Alex Punzano Gil

Juana Valentina Echeveste

Antonio Carbonell Gómez

Pau Cremades García

Compresión y Seguridad (2026-2027)

Trabajo práctico en grupo — Fase I (un único usuario)

Índice

1. Visión general
2. Estructura del proyecto y archivos de arranque
3. Flujo de ejecución de la aplicación
4. Main.java
5. UI.java
6. DbManager.java y formato de database.csv
7. PasswordManager.java
8. RandKeyGenerator.java
9. cypher/AES.java
10. cypher/RSA.java
11. Diseño criptográfico y notación
12. Manejo de errores
13. Pruebas realizadas
14. Limitaciones y mejoras para la Fase II

1. Visión general

La aplicación es un programa de escritorio en Java (Swing) que permite a un usuario **cifrar archivos multimedia** y recuperarlos más tarde. Cada archivo se cifra con **AES-128** usando una clave distinta generada aleatoriamente. Esa clave AES se protege cifrándola con la **clave pública RSA** del usuario, y la **clave privada RSA** se protege cifrándola con AES-128 a partir de la contraseña del usuario.

En la Fase I existe **un único usuario**, llamado u. Solo se pide una contraseña: la primera vez sirve para crear el usuario y, después, para acceder.

Responsabilidades de cada clase

Clase	Responsabilidad
Main	Lógica de negocio: usuario, sesión, cifrar, descifrar, guardado en *cifrados/* y *descifrados/*.
UI	Interfaz gráfica: contraseña, botones Cifrar/Descifrar, drag and drop, informe, apertura automática.
DbManager	Almacén de texto (*database.csv*): guardar y buscar usuario y archivos. No cifra nada.
PasswordManager	Hash SHA3-256 con salt de la contraseña.
RandKeyGenerator	Bytes aleatorios criptográficamente seguros (claves, IV, salt).
cypher.AES	Cifrado y descifrado simétrico AES-128 en modo GCM.
cypher.RSA	Generación de claves RSA-2048, cifrado con clave pública y descifrado con clave privada.

Dependencias

Solo se usa la biblioteca estándar de Java: `javax.crypto` (Cipher, SecretKeySpec, GCMParameterSpec), `java.security` (KeyPair, KeyPairGenerator, KeyFactory, MessageDigest, SecureRandom), `java.nio.file`, `java.util.Base64` y `javax.swing`/`java.awt`. No hay librerías externas.

2. Estructura del proyecto y archivos de arranque

```
.
|-- .gitignore
|-- Iniciar.bat                 Compila y ejecuta sin consola (javaw)
|-- Iniciar(ConConsola).bat     Compila y ejecuta con consola (java)
|-- README.md
|-- cifrados/                   (se crea sola) archivos cifrados  nombre.ext.enc
|-- descifrados/                (se crea sola) archivos recuperados en claro
`-- src/
    |-- code/
    |   |-- Main.java  UI.java  DbManager.java
    |   |-- PasswordManager.java  RandKeyGenerator.java
    |   `-- cypher/  AES.java  RSA.java
    |-- database/database.csv
    `-- img/                    imágenes de prueba (solo funciones de depuración)
```

Iniciar.bat

Compila con `javac src\code\*.java src\code\cypher\*.java`. Si hay error de compilación muestra un mensaje y se detiene. Si no, lanza `javaw -cp src\code Main` (sin ventana de consola). **Debe ejecutarse desde la raíz** porque las rutas de carpetas son relativas.

Iniciar(ConConsola).bat

Igual, pero usa `java` y espera una tecla al terminar, de modo que se ven los mensajes de la consola.

.gitignore

Ignora `*.class`, la configuración de Visual Studio y las carpetas *cifrados/* y *descifrados/* (contenido en claro o cifrado de usuario, que no debe subirse al repositorio).

3. Flujo de ejecución de la aplicación

Arranque

- `Main.main` crea una `UI` y llama a `AbrirVentana()`.
- `AbrirVentana` pide la contraseña (`iniciarSesion`). Si se cancela, el programa termina.
- Si el usuario u no existe, `Main.AccederUsuarioUnico` lo registra con esa contraseña; después hace login.
- Con la sesión iniciada se construye y muestra la ventana principal.

Cifrar

- Selección con explorador (`JFileChooser`, multiselección) o arrastrando (`TransferHandler`).
- Ambos caminos llaman a `UI.cifrarArchivos(lista)`, que en segundo plano (`SwingWorker`) lee cada archivo y llama a `Main.GuardarMiArchivo`.
- `Main.CifrarArchivo` genera clave AES e IV, cifra, cifra la clave con RSA, guarda en *database.csv* y escribe *cifrados/nombre.enc*.

Descifrar

- El botón Descifrar muestra la lista de archivos del usuario (`Main.ListarMisArchivos`).
- Para cada archivo elegido, `Main.GuardarArchivoDescifrado` recupera la clave AES con RSA, descifra con AES y escribe el resultado en *descifrados/*.
- La interfaz abre el archivo con el programa predeterminado del sistema (`Desktop.open`).

4. Main.java

Es el núcleo de la aplicación. Contiene la lógica de usuario y de cifrado y mantiene la sesión activa en variables estáticas. La interfaz solo llama a sus métodos públicos.

4.1 Constantes y variables

Elemento	Tipo	Descripción
DIREC_ARCHIVOS	String	*src/img*. Carpeta de imágenes de prueba; solo la usa `cargarArchivos` (depuración).
DIREC_DESCIFRADOS	String	*descifrados*. Destino de los archivos recuperados.
DIREC_CIFRADOS	String	*cifrados*. Destino de los archivos cifrados (.enc).
USUARIO_UNICO	String	*u*. Nombre del único usuario de la Fase I.
clavePrivadaActiva	PrivateKey	Clave privada RSA del usuario con sesión iniciada.
clavePublicaActiva	PublicKey	Clave pública RSA del usuario con sesión iniciada.
usuarioActivo	String	Nombre del usuario con sesión iniciada (null si no hay).
testKey, testIv, testImgEnc	byte[]	Variables temporales de las funciones de prueba.

4.2 Funciones definitivas de cifrado (las que usa la interfaz)

`public static boolean GuardarMiArchivo(byte[] archivo, String nombre)`

Atajo que cifra con la clave pública de la sesión. Devuelve **false** si ya existe un archivo con ese nombre o el nombre no es válido.

`public static byte[] RecuperarMiArchivo(String nombre)`

Atajo que descifra con la clave privada de la sesión y devuelve el contenido en claro.

`public static String[] ListarMisArchivos()`

Devuelve los nombres de los archivos cifrados del usuario activo (`DbManager.ObtenerArchivos`). Array vacío si no hay ninguno.

`public static Path GuardarArchivoDescifrado(String nombre) throws IOException`

Descifra un archivo y lo escribe en *descifrados/*; devuelve la ruta. Crea la carpeta si no existe. Usa `Paths.get(nombre).getFileName()` para que un nombre con ../ no pueda escribir fuera de la carpeta.

`public static boolean CifrarArchivo(byte[] archivo, PublicKey publicKey, String nombreArchivo)`

Cifra un archivo y lo guarda. Pasos:

- Comprueba que hay sesión (si no, lanza `IllegalStateException`).
- Genera **kf** (16 bytes) e **iv** (12 bytes) con `RandKeyGenerator`.
- `AES.Encode(archivo, kf, iv)` produce el archivo cifrado.
- `RSA.Encode(kf, publicKey)` produce la clave AES cifrada.
- Construye la línea de 5 campos en Base64 [nombre, archivo cifrado, iv, kf cifrada, usuario] y llama a `DbManager.GuardarArchivo`. Si devuelve false (duplicado), termina con false.
- Escribe *cifrados/nombre.enc* con los bytes cifrados. Si falla la escritura en disco solo se avisa por consola; el registro en la base de datos se mantiene.

- Devuelve true.

`public static byte[] DescifrarArcchivo(String nombreArchivo, PrivateKey privateKey)`

(El nombre mantiene la errata original.) Recupera la línea de la base de datos, decodifica Base64, obtiene kf con `RSA.Decode` y descifra con `AES.Decode`. Lanza `IllegalArgumentException` si el archivo no existe **o no pertenece al usuario activo**.

4.3 Gestión de usuario y sesión

`public static boolean ExisteUsuarioUnico()`

true si hay un registro USUARIO con nombre *u* en la base de datos.

`public static boolean AccederUsuarioUnico(String password)`

Si el usuario no existe lo registra con esa contraseña (primer uso); después hace login. Devuelve false si la contraseña es incorrecta. Es el único método que usa la interfaz para entrar.

`public static boolean Register(String userName, String password)`

Crea un usuario. Devuelve false si el nombre está vacío, contiene *;*, la contraseña está vacía o el usuario ya existe.

- Genera el par RSA de 2048 bits (`RSA.GenerarParDeClaves`).
- Genera **salt** de 32 bytes y calcula `CodePass(password, salt)`.
- Genera **iv** de 12 bytes y cifra la clave privada (PKCS#8) con AES-128 usando los 16 primeros bytes del hash.
- Guarda [nombre, clave pública (X.509), clave privada cifrada, salt, iv], todo en Base64.

`public static boolean Login(String userName, String password)`

Devuelve true si el usuario existe y la contraseña es correcta; entonces deja la sesión iniciada.

- Busca el usuario; si no existe devuelve false.
- Recalcula el hash con el salt guardado y descifra la clave privada con AES-GCM. **Una contraseña incorrecta provoca fallo de autenticación y devuelve false.**
- Reconstruye la clave privada con `PKCS8EncodedKeySpec` y la pública con `X509EncodedKeySpec`.
- Solo si todo ha ido bien asigna `usuarioActivo`, `clavePrivadaActiva` y `clavePublicaActiva`.

`private static byte[] ClaveAes128(byte[] hashedPassword)`

Toma los 16 primeros bytes del hash SHA3-256 (32 bytes) para obtener una clave AES de 128 bits, como exige el enunciado.

`public static boolean HaySesion() / public static String UsuarioActivo()`

Consultan el estado de la sesión.

4.4 Funciones de depuración (heredadas)

`cargarArchivos`, `MostrarArchivos`, `CifrarValoresAES` y `DescifrarValoresAES` son pruebas de las primeras versiones: leen las imágenes de *src/img* y comprueban que cifrar y descifrar devuelve el original. **La interfaz ya no las usa**; se conservan como referencia.

5. UI.java

Construye la interfaz con Swing. No contiene criptografía: delega en `Main`. Las operaciones pesadas (leer, cifrar, descifrar) se ejecutan con `SwingWorker` para no congelar la ventana.

`public void AbrirVentana()`

Crea la ventana (700x400, centrada, cierra el programa al cerrarse), llama a `iniciarSesion` y, si tiene éxito, monta: título, botones **Cifrar** y **Descifrar** (panel superior) y una zona de texto con scroll (centro) que muestra los resultados y acepta arrastrar y soltar.

`private boolean iniciarSesion(JFrame ventana)`

Muestra un diálogo con campo de contraseña. Si el usuario no existe indica que es el primer uso (crear contraseña). Repite el diálogo si la contraseña está vacía o es incorrecta. Devuelve false si se cancela.

`private void abrirExploradorYSubir(JFrame, JTextArea)`

Abre un `JFileChooser` con selección múltiple y pasa los archivos elegidos a `cifrarArchivos`.

`private void configurarDragAndDrop(JTextArea)`

Instala un `TransferHandler` que acepta `DataFlavor.javaFileListFlavor` y envía los archivos soltados a `cifrarArchivos`.

`private void cifrarArchivos(List<File>, JTextArea)`

Código común del explorador y del drag and drop. Para cada archivo: rechaza carpetas, lee los bytes y llama a `Main.GuardarMiArchivo`. Construye un informe (cifrado correcto / ya existe / error) y lo muestra al terminar.

`private void descifrarYGuardar(JFrame, JTextArea)`

Si no hay archivos cifrados lo indica. Si los hay, muestra una lista de selección múltiple; para cada elegido llama a `Main.GuardarArchivoDescifrado`, anota la ruta en el informe y **abre el archivo**.

`private void abrirArchivo(Path, StringBuilder)`

Abre el archivo con `Desktop.getDesktop().open` si el sistema lo soporta. Si falla, lo anota en el informe sin interrumpir el resto.

`public void HelloUI()`

Método de prueba heredado; no se usa.

6. DbManager.java y formato de database.csv

Guarda y lee texto en *src/database/database.csv*. **No cifra nada**: recibe datos ya codificados en Base64. Si la carpeta o el archivo no existen, los crea (`AsegurarCsv`).

Formato

```
USUARIO;<nombre>;<clavePublica>;<clavePrivadaCifrada>;<salt>;<iv>
ARCHIVO;<nombreOriginal>;<archivoCifrado>;<iv>;<claveAESCifradaRSA>;<usuario>
```

Siempre hay **6 columnas** separadas por `;`. Las líneas mal formadas se ignoran. Base64 no contiene *;*, así que es seguro como separador.

Método	Función
ObtenerUsuario(userName)	Devuelve los 5 campos del usuario o null.
GuardarUsuario(dataline)	Añade *USUARIO;* + campos unidos por *;*. Devuelve false si el nombre contiene *;* o ya existe.
ObtenerArchivo(nombre)	Devuelve los 5 campos del archivo o null.

Método	Función
GuardarArchivo(dataline)	Igual que usuario, comprobando duplicados por nombre (campo 0).
ObtenerArchivos(userName)	Nombres de todos los archivos de ese usuario (array vacío si no hay).
EscribirLinea(linea)	(privado) Añade una línea al final con `FileWriter(ruta, true)`.
Buscar(tipo, nombre)	(privado) Recorre el CSV y devuelve la primera línea que coincide en tipo y nombre.
BuscarAll(userName)	(privado) Recorre el CSV y recoge los archivos cuyo propietario (columna 6) coincide.
AsegurarCsv()	(privado) Crea carpeta y archivo si faltan.

Actualmente no puede haber dos archivos con el mismo nombre. Para la Fase II, con varios usuarios, la clave de búsqueda debería incluir al propietario.

7. PasswordManager.java

`public static byte[] CodePass(String password, byte[] salt)`

Calcula **SHA3-256(salt || contraseña)**: crea un `MessageDigest` "SHA3-256", introduce el salt con `update` y hace `digest` de la contraseña en UTF-8. Devuelve 32 bytes. El salt evita que dos usuarios con la misma contraseña tengan el mismo hash y frena los ataques con tablas precalculadas. Si el algoritmo no existe lanza `IllegalStateException`. `Main` usa solo los 16 primeros bytes como clave AES-128.

8. RandKeyGenerator.java

`public static byte[] GenKey(int bytes)`

Devuelve *bytes* bytes aleatorios con `SecureRandom` (generador criptográficamente seguro). Se usa para: claves AES (16 bytes), IV de GCM (12 bytes) y salt (32 bytes).

9. cypher/AES.java

Cifrado simétrico con **AES/GCM/NoPadding**. GCM es un modo autenticado: además de cifrar, genera una etiqueta de 128 bits que permite detectar cualquier modificación o el uso de una clave incorrecta.

`public static byte[] Encode(byte[] archivo, byte[] key, byte[] iv)`

Crea `SecretKeySpec(key, "AES")` y `GCMParameterSpec(128, iv)`, inicializa el Cipher en modo cifrar y devuelve `doFinal(archivo)` (texto cifrado + etiqueta de 16 bytes). Con clave de 16 bytes es AES-128.

`public static byte[] Decode(byte[] encArchivo, byte[] key, byte[] iv)`

Operación inversa. Si la clave es incorrecta o los datos fueron alterados, `doFinal` falla y se lanza `IllegalStateException`. `Main.Login` aprovecha esto para detectar una contraseña errónea.

Importante: nunca debe repetirse la pareja (clave, IV) en GCM. Aquí cada archivo tiene su propia clave aleatoria e IV aleatorio, por lo que no ocurre.

10. cypher/RSA.java

Cifrado asimétrico con **RSA/ECB/PKCS1Padding**. Solo se cifran datos muy cortos (la clave AES de 16 bytes), que caben de sobra en un bloque RSA de 2048 bits (256 bytes).

`public static KeyPair GenerarParDeClaves()`

Usa `KeyPairGenerator` "RSA" con 2048 bits y devuelve el par (pública, privada).

`public static byte[] Encode(byte[] claveAES, PublicKey clavePublica)`

Cifra con la clave pública; devuelve 256 bytes.

`public static byte[] Decode(byte[] claveAES, PrivateKey clavePrivada)`

Descifra con la clave privada y devuelve la clave AES original.

Los errores se envuelven en `RuntimeException` con un mensaje descriptivo.

11. Diseño criptográfico y notación

Símbolo	Significado
f, c_f	Archivo en claro y archivo cifrado
k_f	Clave AES-128 del archivo f (aleatoria, una por archivo)
K_u, k_u	Clave pública y privada RSA-2048 del usuario u
p_u	Contraseña del usuario u
s_u	Salt de 32 bytes del usuario u
k_p	Clave AES-128 derivada de la contraseña: 16 primeros bytes de SHA3-256(s_u || p_u)
$E^{AES}_k(m)$, $D^{AES}_k(c)$	Cifrado / descifrado AES-128-GCM con clave k
$E^{RSA}_K(m)$, $D^{RSA}_k(c)$	Cifrado RSA con clave pública K / descifrado con clave privada k

Registro

1) (K_u, k_u) = RSA.GenerarParDeClaves()
2) s_u = aleatorio(32); h = SHA3-256(s_u || p_u); k_p = h[0..15]
3) Se almacena: K_u (en claro), $E^{AES}_{kp}(k_u)$, s_u e iv

Acceso

$k_u = D^{AES}_{kp}(E^{AES}_{kp}(k_u))$. Si p_u es incorrecta, la etiqueta GCM no valida y se deniega el acceso.

Cifrar f

1) k_f = aleatorio(16), iv = aleatorio(12)
2) $c_f = E^{AES}_{kf}(f)$
3) $E^{RSA}_{Ku}(k_f)$
4) Se almacena: c_f, iv, $E^{RSA}_{Ku}(k_f)$

Descifrar f

1) $k_f = D^{RSA}_{ku}(E^{RSA}_{Ku}(k_f))$
2) $f = D^{AES}_{kf}(c_f)$

Qué queda en claro y qué no

Dato	Estado en disco
Archivo f	Cifrado con AES-128-GCM (nunca en claro en *cifrados/* ni en la base de datos)
Clave k_f	Cifrada con RSA; nunca en claro
Clave privada k_u	Cifrada con AES-128 mediante la contraseña; nunca en claro
Contraseña p_u	No se guarda; solo se usa para derivar k_p
Clave pública K_u, salt, IV	En claro (no son secretos)
Archivos recuperados	En claro en *descifrados/* (es el objetivo de descifrar)

12. Manejo de errores

Situación	Comportamiento
Contraseña vacía o incorrecta	Aviso y se vuelve a pedir; cancelar cierra el programa.
Archivo con el mismo nombre ya cifrado	No se guarda; el informe lo indica.
Nombre con *;*	Rechazado (rompería el CSV).
Se intenta cifrar una carpeta	Rechazado con mensaje.
No hay archivos para descifrar	Mensaje informativo.
Archivo inexistente o de otro usuario	`IllegalArgumentException`, mostrada en el informe.
Fallo al escribir en *cifrados/*	Aviso por consola; el registro en la base de datos se conserva.
Fallo al abrir el archivo descifrado	Se anota en el informe; el archivo queda guardado.
Un archivo falla en un lote	Se informa de ese archivo y se continúa con los demás.

13. Pruebas realizadas

Se ejecutó una prueba automatizada (sin interfaz gráfica) sobre una copia del proyecto, que comprobó:

- Primer acceso crea el usuario; segundo acceso con contraseña errónea falla; con la correcta entra.
- Cifrar un binario de 2 MB y un texto; el descifrado es **idéntico byte a byte**.
- Duplicados y nombres con *;* rechazados.
- *cifrados/foto.jpg.enc* existe y es distinto del original.
- El CSV no contiene texto en claro del archivo.
- Un archivo inexistente lanza error; un segundo usuario no puede acceder a los archivos del primero.

La interfaz gráfica (diálogo, botones, drag and drop, apertura automática) compila correctamente pero debe verificarse manualmente en Windows.

14. Limitaciones y mejoras para la Fase II

- **Varios usuarios y compartir archivos:** cifrar k_f con la clave pública del destinatario. Los métodos Register/Login ya admiten nombre de usuario.
- **Derivación de clave:** sustituir el SHA3-256 simple por PBKDF2 (o usar media hash para autenticar y media para la clave privada, como sugiere el enunciado). Valorar 2FA.
- **RSA con OAEP** en lugar de PKCS#1 v1.5.
- **Firma digital e integridad** de archivos.
- **Almacenamiento:** guardar los .enc en carpeta y solo su ruta en el CSV; cifrar por bloques/flujo para archivos de varios GB.
- **Política de contraseñas** (mínimo 16 caracteres, mayúscula y número).
- **Clave de búsqueda de archivos** por (usuario, nombre) para permitir el mismo nombre en usuarios distintos.